GCN, CDNA, RDNA Architecture

Thursday, April 16, 2026 4:27 PM

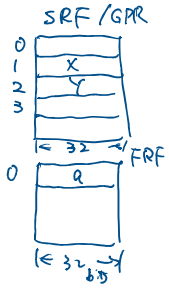
Vector Processor

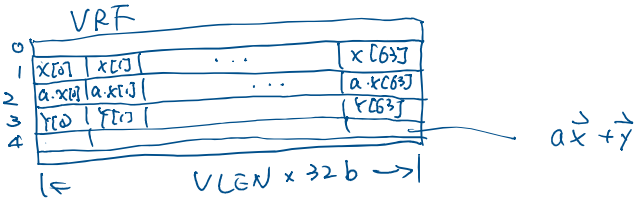
```
L.S F0,a        ;load scalar a
LV V1,R1        ;load vector X
MULVS.D V2,V1,F0 ;vector-scalar multiply
LV V3,R2        ;load vector Y
ADDVV.D V4,V2,V3 ;add
SV R2,V4        ;store the result
```

SAXPY $\vec{Y} = a \cdot \vec{X} + \vec{Y}$

Vector length is 64 (VLEN)

ARF state:





SAXPY on GPU (1 SM) $\vec{Y} = a \cdot \vec{X} + \vec{Y}$ // 64 elements in the vectors.

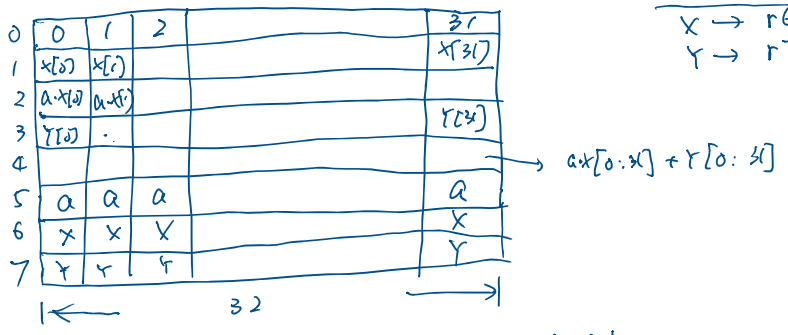
kernel:

```
tid = TBIdx.x * TBDim... + ThreadIdx.x;
Y[tid] = a * X[tid] + Y[tid];
```

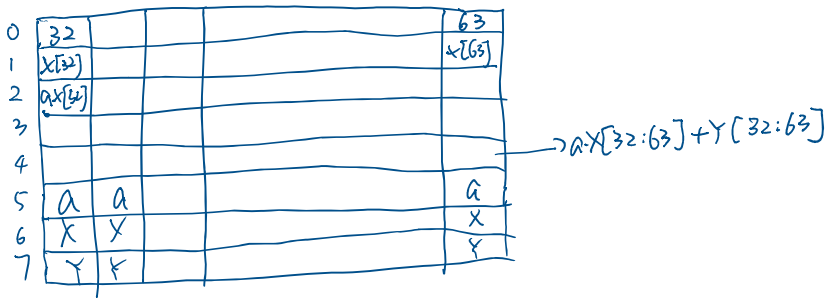
TB size: 64. Grid (1, 1) ⇒ 2 Warps.

tid → r0
x[tid] → r1
a·x[tid] → r2
Y[tid] → r3
aX[id] + Y[tid] → r4.
a → r5
X → r6
Y → r7

Warp 0



Warp 1



↓ Selective Vectorization (ISA → Scalar + Vector instructions)

GCN, CDNA, RDNA.

kernel:

```
tid = TBIdx.x * TBdim... + ThreadIdx.x;
Y[tid] = a * X[tid] + Y[tid];
```

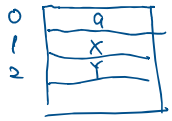
TB size: 64. Grid (1, 1) ⇒ 2 Warps.

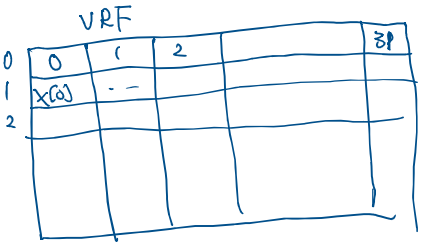
tid → vr0
x[tid] → vr1
a·x[tid] → vr2
Y[tid] → vr3
a x[id] + Y[tid] → vr4.

a → r0
X → r1
Y → r2

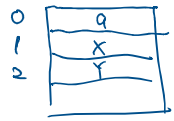
uniform register

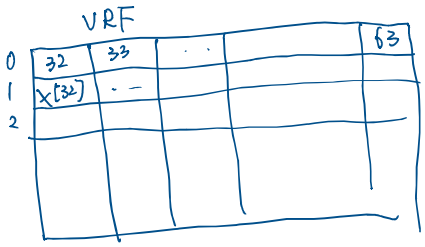
Warp 0

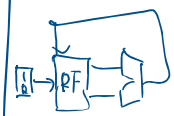
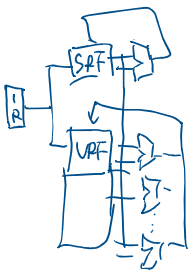
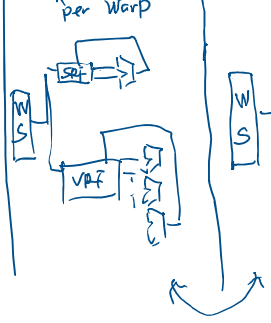
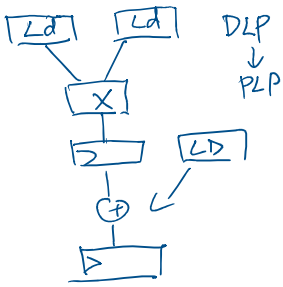




Warp 1





	CPU	Vector Processor	GCN	SIMT	FPGA
Programming Model	Sequential for(i ...) y[i] = a·x[i] + y[i]	Vector $\vec{y} = a \cdot \vec{x} + \vec{y}$	SIMT/CUDA/OPENCL tid = ... y[tid] = a · x[tid] + y[tid]		
ISA	RISC GPRs, FPRs, PC	VRISC GPRs, FPRs, VR, PC SRF, VRF	RISC PCs SRF, VRF per Warp per warp RF		NA
μArch.					

Performance ?

$CPU_{time} = IC * CPI * CT$

Energy Efficiency ?

FPGA

GCN SIMT

← Vector →

CPU